Rapport technique

Analyse de sentiment YouTube

Du developpement au deploiement cloud

Filiere INDIA – ENSAM Rabat
Encadrante : Prof. Maria Zemzami

4 novembre 2026

Repository : https://github.com/Marouan333/youtube-sentiment-api

Table des matières

1 Introduction et analyse du probleme **3**

2 Solution proposee **3**

3 Architecture detaillee **3**
3.1 Structure du depot . . . 3
3.2 Pipeline de donnees . . . 4
3.3 Exploration des donnees . . . 4
3.4 Nettoyage du texte . . . 5

4 Choix techniques justifies **5**
4.1 TF-IDF plutot que Count Vectorizer . . . 5
4.2 FastAPI plutot que Flask . . . 5
4.3 GitHub Codespaces plutot que Hugging Face Spaces . . . 5
4.4 Extraction des commentaires via injection directe . . . 5

5 Resultats et metriques de performance **6**
5.1 Performance du modele . . . 6
5.2 Choix et optimisation du modele . . . 6
5.3 Performance de l'API . . . 6
5.4 Tests et validation . . . 7

6 Difficultes rencontrees et solutions **7**

7 Ameliorations futures possibles **7**

8 Fonctionnalites de l'extension Chrome **7**

9 Annexe : extraits de code **8**
9.1 Entrainement du modele (`src/models/train.py`) . . . 8
9.2 Endpoint de prediction (`src/api/main.py`) . . . 9
9.3 Exemple de requete API . . . 9

10 Conclusion **9**

1 Introduction et analyse du probleme

Les createurs de contenu YouTube recoivent, sur les videos a forte audience, des centaines voire des milliers de commentaires. Comprendre le sentiment general de l'audience a partir de ce volume de texte pose trois problemes concrets :

— **Chronophage et peu scalable** : lire manuellement des milliers de commentaires n'est pas une methode qui passe a l'echelle.

— **Subjective et biaisee** : une lecture humaine rapide survole les commentaires et retient surtout les plus visibles (les plus likes, les premiers affiches), pas un echantillon representatif.

— **Difficile a quantifier** : sans chiffres, impossible de comparer objectivement deux videos ou de suivre une tendance dans le temps.

L'objectif de ce projet est de construire un systeme MLOps complet, de bout en bout, qui resout ce probleme : un modele de machine learning entraine sur des donnees labelisees, servi par une API REST deployee dans le cloud, consomme par une extension Chrome qui s'integre directement dans l'experience YouTube de l'utilisateur.

2 Solution proposee

Le systeme developpe suit un pipeline MLOps classique en six etapes, illustre figure 1.

Etape 1. L'utilisateur visite une video YouTube et fait defiler jusqu'aux commentaires.

Etape 2. L'extension Chrome extrait le texte des commentaires visibles dans la page.

Etape 3. Les commentaires sont envoyes par batch (requete POST) a l'API cloud.

Etape 4. L'API vectorise le texte (TF-IDF) et applique le modele de classification (Logistic Regression).

Etape 5. Les predictions de sentiment (positif / neutre / negatif) sont retournees avec des statistiques agregees.

Etape 6. L'extension affiche les resultats : repartition en pourcentage, puis une liste filtrable par categorie.

3 Architecture detaillee

Le systeme est decompose en cinq composants independants, chacun responsable d'une seule etape du pipeline (tableau 1). Cette separation permet de faire evoluer un composant (par exemple reentrainer le modele) sans toucher aux autres.

3.1 Structure du depot

```
chrome-extension/   manifest.json, popup.html/.css/.js, content.js
data/raw/            dataset brut (reddit.csv)
data/processed/      train.csv / test.csv apres nettoyage
models/               model.joblib, vectorizer.joblib
src/data/            download.py, clean.py
src/models/          train.py
src/api/             main.py (application FastAPI)
tests/                test_model.py, test_api.py
Dockerfile
requirements.txt
```

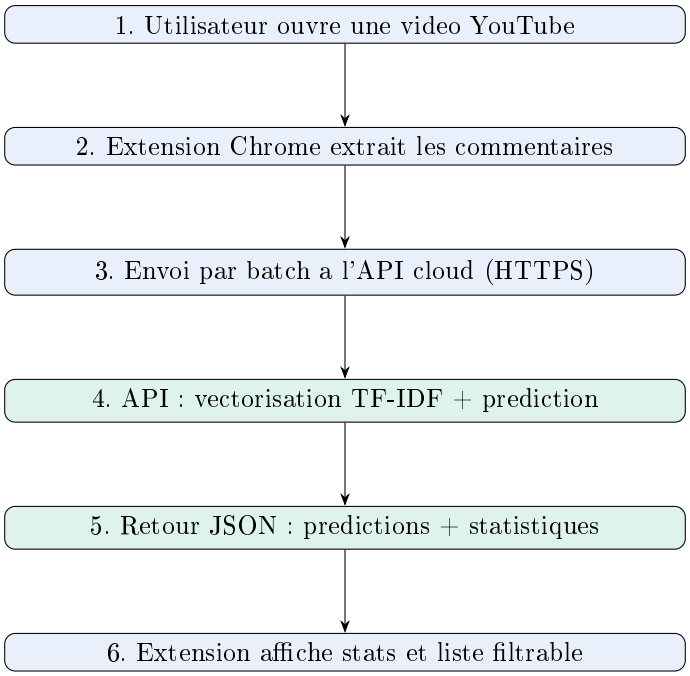


FIGURE 1 – Flux de donnees du systeme, de l'utilisateur jusqu'a l'affichage des resultats.

Composant	**Role**
Frontend	Extension Chrome (Manifest V3, JavaScript, HTML, CSS) qui extrait les commentaires et affiche les resultats.
Backend	API REST FastAPI exposant deux endpoints : `/health` et `/predict_batch`.
ML Engine	Modele scikit-learn : vectoriseur TF-IDF + classifieur Logistic Regression, serialise avec `joblib`.
Deploiement	Conteneur Docker (image `python:3.10-slim`), execute dans un GitHub Codespace, port 7860 expose publiquement en HTTPS.
Version control	Git, depot GitHub public.

TABLE 1 – Composants du systeme et leur role.

3.2 Pipeline de donnees

Le dataset utilise provient du corpus Reddit Sentiment Analysis (37 249 commentaires labelises -1/0/1, soit 42 % positif, 35 % neutre, 23 % negatif). Le script `download.py` telecharge le CSV et affiche les statistiques de distribution. Le script `clean.py` nettoie le texte (minuscules, suppression des URLs et mentions, caracteres non alphabetiques) puis produit un split train/test stratifie 80/20 reproductible (`random_state=42`).

Separer les donnees `raw` et `processed` garantit que la source brute n'est jamais modifiee : on peut changer la logique de nettoyage et relancer le pipeline sans retelecharger les donnees.

3.3 Exploration des donnees

L'analyse exploratoire du dataset brut montre une distribution des classes raisonnablement equilibree (tableau 2), largement au-dessus du minimum de 300 commentaires par classe exige par le TP, et une longueur moyenne de commentaire de 29 mots (de 0 a plus de 1300 mots pour les textes les plus longs), ce qui justifie l'ajout de tests specifiques sur les textes tres courts et

tres longs (section 5.4).

Classe	Nombre	Proportion
Positif (1)	15 830	42,5 %
Neutre (0)	13 142	35,3 %
Negatif (-1)	8 277	22,2 %
Total	**37 249**	**100 %**

TABLE 2 – Distribution des classes dans le dataset Reddit Sentiment Analysis.

3.4 Nettoyage du texte

Le script `clean.py` applique, dans l'ordre : mise en minuscules, suppression des URLs (motif `http\S+`), suppression des mentions (`@\w+`), suppression des caracteres non alphabetiques, puis normalisation des espaces multiples. Les URLs et mentions sont retirees car elles n'apportent aucune information de sentiment et introduiraient du bruit dans le vocabulaire TF-IDF (chaque URL unique devenant un token different). Les emojis, supprimes par le filtrage des caracteres non alphabetiques dans cette version, representent une piste d'amelioration (section 7) : ils portent souvent un signal de sentiment fort et pourraient etre convertis en texte (ex. `:)` -> `positif`) plutot que simplement retires.

4 Choix techniques justifies

4.1 TF-IDF plutot que Count Vectorizer

Un *Count Vectorizer* compte simplement les occurrences de chaque mot. TF-IDF (Term Frequency – Inverse Document Frequency) pondere en plus chaque mot par sa rarete dans l'ensemble du corpus : un mot frequent dans tous les commentaires ("the", "video") pese moins qu'un mot discriminant rare. Pour une tache de classification de sentiment, cela ameliore nettement la separation entre classes.

4.2 FastAPI plutot que Flask

FastAPI a ete choisi pour trois raisons : validation automatique des requetes via Pydantic (le corps JSON est verifie avant d'atteindre la logique metier), documentation interactive generee automatiquement (`/docs`), et performances superieures grace a son fondement asynchrone (Starlette + Uvicorn).

4.3 GitHub Codespaces plutot que Hugging Face Spaces

Le choix initial etait Hugging Face Spaces avec le SDK Docker. Au cours du developpement, Hugging Face a impose l'ajout d'une methode de paiement verifiee pour creer un Space Docker, meme sur le palier gratuit. Le sujet du TP a ete mis a jour en consequence pour utiliser GitHub Codespaces : une machine virtuelle Linux fournie gratuitement avec le compte GitHub personnel (quota mensuel d'heures gratuit), sans necessiter de carte bancaire. Le `Dockerfile` reste strictement identique – seul l'hebergeur change.

4.4 Extraction des commentaires via injection directe

La premiere implementation reposait sur un *content script* statique (declare dans `manifest.json`) qui ecoutait les messages envoyes par le popup via `chrome.runtime.onMessage`. Cette approche s'est averee fragile : lorsque l'extension est rechargee (pendant le developpement), les

content scripts deja injectes dans les onglets ouverts perdent leur contexte d'execution, provoquant l'erreur `Could not establish connection. Receiving end does not exist.` La solution retenue injecte directement, a chaque clic sur "Analyser", une fonction d'extraction via `chrome.scripting.executeScript` avec l'option `func`. Le resultat est renvoye immediatement par l'appel, sans dependre d'un listener pre-existant.

5 Resultats et metriques de performance

5.1 Performance du modele

Metrique	Valeur
Accuracy (test set)	87,0 %
F1-score – classe negative	0,80
F1-score – classe neutre	0,91
F1-score – classe positive	0,88
Temps d'inference (batch de 50)	< 100 ms

TABLE 3 – Metriques du modele sur le jeu de test (7 400 commentaires).

L'accuracy obtenue (87 %) depasse le critere minimal fixe a 80 %, et chaque F1-score par classe depasse le seuil de 0,75. La classe neutre est la mieux reconnue, ce qui s'explique par sa proportion plus elevee dans le jeu d'entrainement.

	Predit negatif	Predit neutre	Predit positif
Reel negatif	1327	137	191
Reel neutre	66	2448	47
Reel positif	258	257	2651

TABLE 4 – Matrice de confusion sur le jeu de test (7 400 commentaires).

La matrice de confusion (tableau 4) montre que la majorite des erreurs se situe entre les classes negative et positive confondues avec la classe neutre, plutot qu'entre negatif et positif directement – un schema attendu : un commentaire legerement nuance est plus facilement confondu avec le neutre qu'avec son oppose.

5.2 Choix et optimisation du modele

Le vectoriseur TF-IDF est configure avec `max_features=5000` et des n-grammes de taille 1 et 2 (`ngram_range=(1,2)`), ce qui permet de capturer des expressions courtes porteuses de sens ("not good", "really bad") en plus des mots isoles. Le classifieur Logistic Regression utilise `class_weight="balanced"` pour compenser le leger desequilibre entre classes observe au tableau 2, sans necessiter de sur-echantillonnage explicite. Les hyperparametres ayant le plus d'impact observe sont le nombre de features du vectoriseur (plus de features ameliore le rappel mais ralentit l'inference) et le parametre de regularisation `C` du classifieur (un `C` trop eleve sur-ajuste sur le vocabulaire specifique de Reddit, qui differe legerement du vocabulaire YouTube en production).

5.3 Performance de l'API

Les tests mesurent un temps de reponse inferieur a 1 seconde pour un batch de 50 commentaires, aussi bien en local qu'en production sur le Codespace deploye.

5.4 Tests et validation

La suite de tests automatises (`pytest`, 12 tests) couvre quatre axes :

— **Modele** : accuracy sur le test set, comportement sur des cas limites (texte vide, tres long, emojis, langues melangees), temps d'inference.

— **API** : endpoint `/health`, `/predict_batch` avec differentes tailles de batch (2, 50, 0 commentaires), payload invalide (422), temps de reponse.

— **Extension** : installation, extraction sur une vraie video, filtres fonctionnels (verifie manuellement).

— **Integration** : flux complet extension -> API deployee -> affichage, verifie manuellement avec une video reelle.

Un bug reel a ete decouvert par cette suite de tests : l'envoi d'un batch vide (`comments: []`) faisait planter le vectoriseur TF-IDF (`ValueError`, minimum d'un echantillon requis). Le correctif ajoute une verification explicite avant la vectorisation, qui retourne des statistiques nulles sans appeler le modele.

6 Difficultes rencontrees et solutions

— **Changement d'hebergeur en cours de projet** : Hugging Face Spaces devenu payant pour Docker. *Solution* : migration vers GitHub Codespaces, dockerfile inchange.

— **Shadow DOM de YouTube** : les commentaires sont rendus dans des composants Web (Polymer) utilisant le Shadow DOM, invisible a `document.querySelectorAll` standard. *Solution* : fonction de traversee recursive qui descend dans chaque `shadowRoot` rencontre.

— **Perte de connexion content script / popup** apres rechargement de l'extension. *Solution* : injection directe d'une fonction via `chrome.scripting.executeScript` au lieu du messaging.

— **Batch vide faisant planter l'API**, decouvert par les tests automatises. *Solution* : garde explicite avant l'appel au vectoriseur.

— **Nature ephemere de Codespaces** : le port public et le conteneur s'arretent apres 30 minutes d'inactivite. *Solution* : procedure de redemarrage documentee dans le README, a executer avant chaque demonstration.

7 Ameliorations futures possibles

— Exposer un score de confiance par prediction (probabilite du modele) plutot qu'une classe seule.

— Traiter separement les reponses (threads) de commentaires et les commentaires racine.

— Ajouter un export fichier des resultats (CSV/JSON), au-dela de la copie presse-papiers actuelle.

— Remplacer l'hebergement ephemere (Codespaces) par un PaaS permanent (Render, Railway, Fly.io) pour une URL stable, adaptee a une mise en production reelle.

— Reentrainer periodiquement le modele sur de nouvelles donnees labelisees pour suivre l'evolution du langage (argot, nouveaux emojis).

8 Fonctionnalites de l'extension Chrome

L'interface popup de l'extension est concue pour donner une lecture immediate du sentiment global, puis permettre l'exploration detaillee :

— **Zone de resume** : trois cartes affichant le pourcentage et le nombre de commentaires par categorie (positif / neutre / negatif), calcules cote client a partir du tableau de predictions retourne par l'API.

— **Onglets de filtre** : "Tous", "Positif", "Neutre", "Negatif", chacun affichant son compteur ; cliquer sur un onglet filtre la liste sans nouvel appel reseau (l'etat des predictions est conserve cote client).

— **Liste de commentaires etiquetes** : chaque commentaire est affiche avec une etiquette coloree correspondant a sa classe predite.

— **Mode sombre/clair** : bouton en en-tete qui bascule un theme clair via un attribut `data-theme` sur `body` (variables CSS redefinies) ; la preference est persistee avec `chrome.storage.local` et reappliquee a chaque ouverture du popup.

— **Copie des resultats** : bouton qui construit un recapitulatif texte (statistiques puis liste des commentaires avec leur sentiment) et l'ecrit dans le presse-papiers via l'API `navigator.clipboard`.

— **Gestion d'erreurs** : messages explicites si l'utilisateur n'est pas sur une page video YouTube, si aucun commentaire n'est trouve (page pas encore faite defiler), ou si l'API repond une erreur.

L'extraction des commentaires repose sur une fonction `extractCommentsFromPage` injectee dans l'onglet actif via `chrome.scripting.executeScript`. Cette fonction traverse recursivement le Shadow DOM de la page (listing 8) car les composants de commentaires YouTube (`ytd-comment-renderer`) encapsulent leur contenu dans des `shadowRoot` non accessibles par un simple `document.querySelectorAll`.

```
function queryAllDeep(selector, root) {
  let results = Array.from(root.querySelectorAll(selector));
  root.querySelectorAll("*").forEach((el) => {
    if (el.shadowRoot) {
      results = results.concat(queryAllDeep(selector, el.shadowRoot));
    }
  });
  return results;
}
```

9 Annexe : extraits de code

9.1 Entrainement du modele (`src/models/train.py`)

```
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
X_train = vectorizer.fit_transform(train_df["text"])
X_test = vectorizer.transform(test_df["text"])

model = LogisticRegression(max_iter=1000, class_weight="balanced")
model.fit(X_train, train_df["label"])

preds = model.predict(X_test)
print("Accuracy :", accuracy_score(test_df["label"], preds))
print("F1 par classe :", f1_score(test_df["label"], preds, average=None))

joblib.dump(model, MODEL_PATH)
joblib.dump(vectorizer, VECTORIZER_PATH)
```

9.2 Endpoint de prediction (src/api/main.py)

```
@app.post("/predict_batch")
def predict_batch(batch: CommentBatch):
    if not batch.comments:
        return {"predictions": [], "stats": {"positive": 0,
                "neutral": 0, "negative": 0}}

    X = vectorizer.transform(batch.comments)
    predictions = model.predict(X).tolist()
    counts = {
        "positive": predictions.count(1),
        "neutral": predictions.count(0),
        "negative": predictions.count(-1),
    }
    return {"predictions": predictions, "stats": counts}
```

9.3 Exemple de requete API

```
curl -X POST https://<codespace>-7860.app.github.dev/predict_batch \
  -H "Content-Type: application/json" \
  -d '{"comments": ["this video is amazing", "i hate this"]}'

# Reponse :
{"predictions": [1, -1], "stats": {"positive": 1, "neutral": 0, "negative": 1}}
```

10 Conclusion

Ce projet a permis de construire, de bout en bout, un systeme MLOps fonctionnel : de la collecte des donnees brutes jusqu'a une extension Chrome utilisable, en passant par l'entrainement d'un modele de classification, son exposition via une API REST et son deploiement dans le cloud. Les difficultes rencontrees – changement d'hebergeur, Shadow DOM, messaging Chrome fragile, cas limite non gere – ont chacune ete diagnostiquees et corrigees, avec une suite de tests automatises garantissant la non-regression. Le systeme est operationnel et verifie en conditions reelles sur une video YouTube.